QUALITÉ DE DÉVELOPPEMENT

DÉVELOPPEMENT COLLABORATIF AVEC GIT ET GITLAB

R2.03

INTRODUCTION : LE PROBLÈME

TRAVAILLER SEUL SUR UN PROJET

Imaginons un étudiant qui développe un programme :

```
projet/
├── main.c
└── utils.c
```

Il modifie son programme régulièrement.
Comment sauvegarder l’évolution ?

SOLUTION NAÏVE

Créer des copies du projet :

```
projet/
projet_v1/
projet_v2/
projet_final/
projet_final_corrige/
projet_final_corrige_v2/
```

Problèmes :

- impossible de savoir **ce qui a changé**
- impossible de revenir **facilement à une version précise**
- beaucoup de **duplication de fichiers**
- difficile de comprendre **l'historique**

SI PLUSIEURS PERSONNES TRAVAILLENT ENSEMBLE ?

Imaginez maintenant :

- Alice modifie `main.c`
- Bob modifie `utils.c`

Ils échangent leurs fichiers par :

- mail
- clé USB
- Discord
- Google Drive

Problèmes :

- qui a la **bonne version** ?
- comment **fusionner les modifications** ?
- que faire si **deux personnes modifient la même ligne** ?

LE BESOIN : UN GESTIONNAIRE DE VERSION

Un **Version Control System (VCS)** permet de :

- sauvegarder **l'historique d'un projet**
- voir **les différences entre versions**
- revenir à **une version précédente**
- travailler **à plusieurs**
- gérer les **conflits de modifications**
- collaborer **plus efficacement**

POURQUOI GIT ?

DÉFINITION

Git est un **système de gestion de versions distribué**.
Il permet de :

- **suivre l'évolution** d'un projet
- **travailler à plusieurs**
- conserver l'**historique des modifications**
- gérer les **branches de développement**
- **fusionner les changements**
- **résoudre les conflits**
- … et bien plus encore !

HISTORIQUE DES VCS

Avant Git :

- CVS
- BitKeeper
- Subversion (SVN)

De part leur architecture centralisée, ces systèmes avaient des limitations :

- performance
- gestion des branches
- collaboration

De plus, certains étaient propriétaires (ex : BitKeeper), ce qui posait des problèmes de contrôle et de confiance, surtout pour un projet aussi important que le noyau Linux qui se fait porte étendard de l’open source.

HISTOIRE DE GIT

Git a commencé à être créé en 2005 par Linus Torvalds pour gérer le développement du noyau Linux.
Les raisons de sa création :

- insatisfaction avec les outils existants
- besoin d'un système rapide, fiable et flexible
- volonté de garder le contrôle sur le développement du noyau
- besoin de gérer un grand nombre de contributeurs qui travaillent de manière asynchrone

Git s'est rapidement imposé comme le VCS de référence dans le monde du développement logiciel.

> ℹ️
> Comme l'explique Linus Torvalds dans le premier commit de Git, le projet se nomme ainsi car "git" peut signifier plusieurs chose en fonction du contexte. Il a choisi ce nom de manière humoristique, en se décrivant lui-même comme un "git" ("idiot" en argot britannique) pour avoir créé un système de gestion de versions.
> Lien vers le fichier README du commit original :
> **Github - git/git - README**

PRINCIPES DE GIT

Sa création repose sur 3 principes fondamentaux :

PERFORMANCE

Git utilise des algorithmes efficaces pour stocker et récupérer les données, ce qui permet de gérer des projets avec des milliers de fichiers et des millions de commits sans ralentissement significatif.

SÉCURITÉ

Git garantit l'intégrité des données grâce à l'utilisation de hachages cryptographiques (SHA-1) pour identifier les commits, les branches et les fichiers. Cela permet de détecter toute modification non autorisée ou corruption des données.

FLEXIBILITÉ

Git offre une grande flexibilité dans la gestion des branches et des workflows de développement. Git prend également en charge différents modèles de développement, tels que Git Flow, GitHub Flow ou GitLab Flow (pour ne citer qu'eux), permettant aux équipes de choisir celui qui convient le mieux à leur

processus de travail.

AVANTAGES DE GIT

Git permet :

- un **historique complet**
- des **branches légères**
- un travail **offline**
- une collaboration **facilitée**
- une **communauté active** et de nombreux outils intégrés
- une **intégration avec d'autres outils**
- une **grande flexibilité** dans les workflows de développement
- une **sécurité renforcée** grâce à l'utilisation de hachages cryptographiques
- une **performance élevée** même pour les grands projets
- une **courbe d'apprentissage progressive** grâce à une documentation abondante et une communauté active
- une **compatibilité avec d'autres VCS** grâce à des outils de migration et de conversion
- ... et encore plein d'autre fonctionnalités !

ARCHITECTURE DE GIT

DÉPÔT GIT

Un projet Git est appelé **repository (repo)**.
Il contient à la fois **les fichiers du projet** et **l'historique de leurs modifications**.
Le stockage est optimisé pour la performance et la flexibilité grâce à une structure de données appelée Graphe orienté acyclique, en anglais Directed Acyclic Graph ou DAG.
Tous les fichiers nécessaires au fonctionnement de Git sont stockés dans un dossier caché `.git` à la racine du projet.

DÉPÔT LOCAL ET DÉPÔT DISTANT

De part son architecture distribuée, Git utilise deux types de dépôts : local et distant. Cette distinction est importante car à contrario d'un système centralisé, Git permet à chaque utilisateur de disposer d'une copie complète du projet, et d'y apporter des modifications de manière indépendante avant de les partager avec les autres.

DÉPÔT LOCAL

Ce type de dépôt est stocké sur votre ordinateur. C'est là que vous faites vos modifications, vos commits, et où vous gérez votre historique localement.

DÉPÔT DISTANT

Ce type de dépôt est stocké sur une autre machine, souvent un serveur ou une plateforme en ligne (ex : GitHub, GitLab). C'est là que vous partagez votre travail avec les autres et où vous collaborez.
Git est agnostique du protocole de communications utilisé mais les plus courants sont HTTP/HTTPS et SSH.



> Git est prévu pour fonctionner de pair à pair (peer-to-peer) donc un dépôt distant n’est pas nécessairement centralisé, il peut être hébergé sur un serveur ou une plateforme en ligne, mais il peut aussi être un dépôt sur un autre ordinateur de votre réseau local, ou même sur une clé USB.

LES TROIS ZONES DE GIT

Git fonctionne avec **trois zones principales** :

WORKING DIRECTORY

La zone de travail où sont modifiés les fichiers du projet. Les changements ne sont pas encore suivis par Git.
C'est littéralement ce que vois votre système de fichiers.

STAGING AREA

La zone de préparation où sont sélectionnés les changements à inclure dans le prochain commit. C'est une étape intermédiaire qui permet de préparer un commit propre et cohérent.
C'est une zone virtuelle gérée par Git, elle n'existe pas physiquement sur le disque, mais elle est représentée par un index dans le dossier `.git`.

REPOSITORY

La zone de stockage pérmanente où sont enregistrés les commits, c'est là que Git conserve l'historique complet du projet. C'est un ensemble de

fichiers et de dossiers dans le dossier `.git`.

COMMIT

Un **commit** est un **instantané du projet** à un moment donné. Il contient entre autres :

- les fichiers du projet
- un message de commit
- un auteur
- une date
- un parent (sauf pour le commit initial)
- un identifiant unique (SHA-1)

Chaque commit est lié à son parent, formant ainsi une chaîne d’historique.

BRANCHES

Une **branche** est une **pointe mobile** qui pointe vers un commit spécifique. Elle permet de travailler sur différentes versions du projet en parallèle. Par défaut, Git crée une branche appelée `main` (anciennement `master`) qui pointe vers le commit initial.
Ainsi, lorsque vous faites un nouveau commit, la branche sur laquelle vous travaillez avance pour pointer vers ce nouveau commit, tandis que les autres branches restent inchangées.

> ℹ️
> Une branche n'est ni plus ni moins qu'un pointeur vers un commit, elle ne contient pas de données en elle-même, c'est le commit qui contient les données.

GIT EN PRATIQUE

COMMANDES DE BASE — CONFIGURATION GLOBALE

CONFIGURER GIT

Avant de commencer, il est important de configurer Git avec votre nom et votre adresse e-mail :

```
git config --global user.name "NOM Prénom"
git config --global user.email
"votre.email@domaine.com"
```

Cette configuration est utilisée pour identifier l'auteur de chaque commit dans l'historique du projet.

> 📝
> Réglez avec le même nom (« NOM Prénom ») que celui qui vous est attribué sur le GitLab AMU.

Il est également recommandé de configurer votre éditeur de texte préféré pour les messages de commit :

```
git config --global core.editor "code --wait"
```

Cette commande configure Visual Studio Code comme éditeur de texte pour Git. Vous pouvez remplacer `code --wait` par la commande correspondant à votre éditeur préféré (par exemple,

`nano`, `vim`, etc.).

> ℹ️
> Git est très configurable : vous pouvez personnaliser de nombreux aspects de son comportement en utilisant la commande `git config`. Autant globalement à la machine qu'au niveau d'un projet spécifique.

COMMANDES DE BASE — DÉMARRER ET COMPRENDRE L'ÉTAT

INITIALISER UN PROJET GIT

Créer un dépôt Git dans le dossier courant :

```
git init
```

> ℹ️
> Cette commande crée un dossier caché `.git` : c'est là que Git stocke l'historique. À partir de maintenant, Git peut suivre les modifications du répertoire.

VÉRIFIER L'ÉTAT DU DÉPÔT

Voir "où vous en êtes" (très utile) :

```
git status
```

Montre notamment :

- quels fichiers ont été **modifiés**
- quels fichiers sont prêts à être enregistrés (**staged**)
- quels fichiers sont **non suivis** (nouveaux)

COMMANDES DE BASE - ENREGISTRER UNE ÉTAPE

AJOUTER AU STAGING (PRÉPARER LE PROCHAIN COMMIT)

Sélectionner un fichier à inclure dans le prochain commit :

```
git add hello.txt
```

Ou sélectionner *tous* les changements du dossier :

```
git add .
```

> ℹ️
> Le **staging** (zone d'index) sert à choisir précisément ce qui va entrer dans le prochain commit. Ça évite de commiter "tout et n'importe quoi".

CRÉER UN COMMIT (ENREGISTRER DANS L'HISTORIQUE)

Créer une "étape" :

```
git commit -m "Ajout du fichier hello"
```

Un commit = une étape claire.

Essayez de faire des commits **petits** et avec un message **compréhensible**.

COMMANDES DE BASE — LIRE L'HISTORIQUE ET COMPARER

VISUALISER L'HISTORIQUE

```
git log
```

Variante plus lisible en démo :

```
git log --oneline --graph --decorate
```

VOIR LES DIFFÉRENCES

Voir ce qui a changé **avant** de préparer le commit :

```
git diff
```

Voir ce qui est déjà en staging :

```
git diff --staged
```

BRANCHES

Une **branche** permet de travailler sur une fonctionnalité/correction **à part**, sans impacter directement la branche principale.
Créer une branche et se déplacer dessus :

```
git switch -c feature/hello
```

Voir les branches :

```
git branch
```

Revenir à la branche principale (selon le projet : `main` ou `master`) :

```
git switch main
```

FUSION (MERGE)

La fusion (merge) permet de regrouper le travail de deux branches.
Une fois la fonctionnalité terminée, on **fusionne** la branche dans la principale :

```
git switch main
git merge feature/hello
```

> ℹ️
> La fusion regroupe l'historique et le contenu.
> Parfois Git n'arrive pas à fusionner automatiquement : c'est un **conflit** que l'on verra plus tard.

SYNCHRONISER AVEC UN DÉPÔT EN LIGNE

- envoyer votre travail : `git push`
- récupérer le travail des autres : `git pull`
- récupérer le travail des autres sans fusionner : `git fetch`

> 📝
> La collaboration se fait souvent via une *Merge Request / Pull Request* :
> on propose une branche, on relit, puis on fusionne.

GESTION DES CONFLITS

QU'EST-CE QU'UN CONFLIT GIT ?

Un **conflit** se produit lorsque Git **n'arrive pas à fusionner automatiquement** des modifications, car deux versions concurrentes touchent **la même zone** d'un fichier (souvent les mêmes lignes, ou des lignes très proches).

Git sait fusionner automatiquement quand :

- les modifications portent sur **des lignes différentes**
- ou quand il peut déterminer sans ambiguïté l'ordre des changements

Git est bloqué quand :

- deux personnes modifient **la même ligne**
- une personne modifie une zone que l'autre a **supprimée**
- deux branches font des modifications incompatibles dans une zone trop proche

SITUATION 1 — MODIFICATION DE LA MÊME LIGNE

Version originale :

```
int max = 10;
```

Alice :

```
int max = 20;
```

Bob :

```
int max = 50;
```

Git ne peut pas deviner quelle valeur est correcte : **c'est une décision fonctionnelle**, pas technique.

CE QUE GIT FAIT EN CAS DE CONFLIT

Quand une fusion rencontre un conflit, Git **modifie le fichier** en y insérant des **marqueurs (balises) de conflit** pour montrer clairement les deux versions. Exemple typique :

```
<<<<<<< HEAD
int max = 20;
=======
int max = 50;
>>>>>>> branch
```

- `<<<<<<< HEAD` : début de la version **actuelle** (la branche sur laquelle vous êtes)
- `=======` : séparation entre les deux versions
- `>>>>>>> branch` : fin de la version **qui arrive** (branche fusionnée)

  À ce stade :
- le fichier est **dans un état invalide** (ex: code C non compilable)
- Git passe en état “merge en cours”
- un `git status` indique quels fichiers sont en conflit

RÉSOUDRE UN CONFLIT

Résoudre un conflit, c'est :

1. **choisir** ce qu'on veut garder (version A, version B, ou une combinaison)
2. **éditer le fichier** pour enlever les marqueurs et obtenir un contenu correct
3. **valider** la résolution

RÉSOLUTION MANUELLE

Par exemple, on décide que la bonne valeur est `50` :

```
int max = 50;
```

Ou on combine (exemple : prendre le maximum des deux ou clarifier la variable) :

```
int max = 50; // valeur décidée après discussion / règle métier
```

Une fois le fichier corrigé (sans `<<<<<<<`, `=======`, `>>>>>>>`) :

```
git add fichier
git commit
```

SITUATION 2 — MODIFICATION VS SUPPRESSION

Version originale :

```
timeout=30
```

Alice modifie :

```
timeout=60
```

Bob supprime la ligne (ou supprime le fichier).
Git ne sait pas si la ligne devait exister ou non : conflit possible. La résolution consiste à décider :

- on garde la ligne (et laquelle ?)
- ou on la supprime vraiment

OUTILS MODERNES : RÉSOLUTION ASSISTÉE PAR IDE

Même si on peut tout faire “à la main” en éditant le fichier, **la plupart des IDE** (VS Code, IntelliJ, Eclipse, etc.) intègrent un outil de résolution de conflits :

- affichage **côte à côte** (votre version / leur version /

- boutons du type :
  - *Accepter la version actuelle*
  - *Accepter la version entrante*
  - *Accepter les deux*
  - *Comparer / éditer le résultat*
- prévisualisation du résultat final
- navigation rapide entre conflits

  Ces outils réduisent fortement les erreurs (ex : oublier un marqueur) et accélèrent la résolution, mais le principe reste le même : **vous décidez du contenu final**.

BONNES PRATIQUES DE COMMIT

POURQUOI “BIEN COMMITER” ?

Un commit, ce n’est pas juste “sauvegarder” : c’est une **unité d’historique**. Un bon historique permet de :

- comprendre **ce qui a changé** et **pourquoi**
- retrouver l’origine des changements
- relire un projet comme une suite d’étapes logiques
- faciliter les revues de code

FAIRE DES COMMITS PETITS ET COHÉRENTS

Un commit doit idéalement :

- faire **une seule chose** (une intention)
- être **facile à relire**
- être **facile à annuler** (`git revert`) ou à déplacer (rebase/cherry-pick)

Exemples de “bons” commits :

- “Ajout validation email côté client”
- “Correction du calcul de TVA”
- “Refactor : extraire une fonction de parsing”

Exemples de commits à éviter :

- “test”
- “plein de trucs”
- “fix + refactor + ajout feature + formatage” dans un seul commit

> 📝
> Astuce : si votre commit touche **beaucoup de fichiers non liés**, c’est souvent un signe qu’il est trop gros.

COMMITER SOUVENT, MAIS PAS N'IMPORTE COMMENT

Bon rythme :

- plusieurs commits par fonctionnalité
- plusieurs commits par jour
- éviter le "gros commit final" difficile à relire

Objectif : pouvoir relire l'historique et se dire :

> *"Chaque commit apporte une étape compréhensible."*

UN BON MESSAGE DE COMMIT, C'EST QUOI ?

Un message de commit sert à répondre à 2 questions :

1. **Qu'est-ce qui change ?**
2. **Pourquoi ça change ?** (souvent plus important)

Un bon message :

- est **court et précis**
- décrit l'**intention**
- donne du contexte si nécessaire (bug, ticket, contrainte, impact)

UN TITRE (OBLIGATOIRE)

- une seule ligne
- clair, actionnable
- décrit le changement principal
  Exemples :

```
fix: corrige le calcul du score en cas d’égalité
```

```
feat: ajoute l’export CSV des résultats
```

```
refactor: extrait la logique de parsing dans un
module dédié
```

UN CORPS (OPTIONNEL MAIS TRÈS UTILE)

À utiliser quand “le titre ne suffit pas”. On y met :

- le **contexte** (problème initial)
- la **raison** du choix technique
- les **effets de bord** / impacts

Exemple :

```
fix: corrige le calcul du score en cas d’égalité

Le score était incrémenté deux fois lorsque deux
joueurs avaient le même temps.
On applique maintenant une règle déterministe
basée sur l’ordre d’inscription.
```

FOOTER / RÉFÉRENCES (OPTIONNEL)

- lien vers une issue / ticket
- breaking changes (si pertinent)
  Exemple :

```
fix: empêche le double comptage des points
Refs: #42
```

EXEMPLES : MAUVAIS VS BONS MESSAGES

Mauvais :

```
fix
```

```
update
```

```
modifs
```

Bons :

```
fix: empêche un crash quand le fichier de config
est absent
```

```
docs: clarifie la procédure d’installation sous
Linux
```

```
test: ajoute des tests sur le tri des résultats
```

```
chore: met à jour les dépendances de sécurité
```

CONVENTION (OPTIONNELLE) : CONVENTIONAL COMMITS

Pour avoir des messages homogènes, de nombreux projets suivent **Conventional Commits**. Ce n'est **pas obligatoire**, mais c'est pratique pour :

- standardiser les messages
- générer des changelogs automatiquement
- clarifier rapidement la nature du commit

Format (idée générale) :

```
<type>[optional scope]: <description>
```

Types fréquents (exemples) :

- `feat` : nouvelle fonctionnalité
- `fix` : correction de bug
- `docs` : documentation
- `test` : tests
- `refactor` : refactorisation (sans ajout de feature)
- `chore` : tâches "maintenance" (dépendances, tooling, etc.)

BONNES PRATIQUES RAPIDES (CHECKLIST)

Avant de commiter, vérifiez :

- ▢ le commit correspond à **une intention**
- ▢ le message explique **quoi** (titre) et si besoin **pourquoi** (corps)
- ▢ vous n’avez pas embarqué des changements “parasites” (formatage, fichiers non liés)
- ▢ les tests/build passent (si applicable)

ET DANS LA VRAI VIE ?

Dans la vraie vie, on fait souvent des commits imparfaits (c'est normal), mais l'idée est de viser une **amélioration continue** : petit à petit, essayer de faire mieux, d'avoir un historique plus clair, plus facile à relire et à maintenir.
Dans la plupart des projets "professionnels", d'autre politiques liées aux commits peuvent s'appliquer, comme :

- **Squash** : fusionner plusieurs commits liés en un seul avant de les intégrer à la branche principale (ex: lors d'une Merge Request) pour garder un historique plus propre.
- **Rebase** : réécrire l'historique pour intégrer les changements de la branche principale avant de fusionner, afin d'éviter les commits de merge et garder un historique linéaire.
  Ce sont des actions plus avancées de Git, mais qui permettent de maintenir un historique plus lisible et cohérent, surtout dans les projets avec beaucoup de contributeurs.

RÉCAPITULATIF GIT

C’EST QUOI GIT ?

Git est un outil qui permet de **garder l’historique** d’un projet (souvent du code). Au lieu d’avoir “plein de versions” de fichiers, on enregistre des étapes appelées **commits**.
Un commit, c’est un instantané du projet à un moment donné, avec :

- un auteur
- une date
- un message qui explique le changement
  Git fonctionne sur votre machine **en local**, et on peut aussi envoyer/récupérer le travail via un dépôt en ligne pour effectuer du travail collaboratif.

À QUOI ÇA SERT ?

Git permet notamment de :

- **Sauvegarder l'évolution du projet**
  Retrouver ce qui a été modifié et à quel moment.
- **Revenir en arrière**
  Si on casse quelque chose, on peut revenir à une version précédente.
- **Travailler en équipe**
  Chacun peut avancer de son côté, puis on regroupe le travail.
- **Utiliser des branches**
  Une branche sert à travailler sur une idée (fonctionnalité, correction) sans toucher directement au "travail principal".
- **Fusionner le travail**
  Quand on regroupe deux versions, Git peut parfois hésiter : c'est un **conflit**, et il faut choisir quoi garder.
- **Partager / centraliser le projet**
  Le dépôt en ligne sert de point de partage, de sauvegarde et souvent d'endroit pour faire des revues et bien plus encore.

MOTS-CLÉS À RETENIR

- **Commit** : une étape dans l'historique du projet
- **Branch (branche)** : une "copie" de travail pour développer sans impacter le reste
- **Merge (fusion)** : regrouper le travail de deux branches
- **Remote (distant)** : dépôt en ligne
- **Conflit** : Git ne peut pas fusionner tout seul → décision humaine

PLATEFORME COLLABORATIVE

GIT ≠ GITLAB

- **Git** est un outil de versioning qui fonctionne **sur votre machine** (en local).
- **GitLab** (comme GitHub / Bitbucket) est une **plateforme en ligne** qui héberge des dépôts Git et fournit des outils pour travailler **à plusieurs**.
  En résumé :
- Git = gérer le versioning du projet
- GitLab = partager, collaborer, relire, organiser

CE QUE GITLAB APPORTE CONCRÈTEMENT

GitLab est :

- un **hébergeur Git** (le dépôt distant, “remote”)
- une **plateforme collaborative** (revues, discussions, organisation)

Ça permet notamment :

- centraliser le code (sauvegarde + point de référence)
- travailler en équipe sans s’envoyer des fichiers
- relire et valider les changements avant de les intégrer
- suivre l’avancement (tickets, discussions)

WORKFLOW SIMPLE 1/2

RÉCUPÉRER LE PROJET

```
git clone URL
```

Commande à ne jouer qu’une seule fois pour récupérer le projet sur votre machine. C’est à partir de là que vous allez faire vos modifications, commits, etc.

WORKFLOW SIMPLE 2/2

CYCLE DE TRAVAIL CLASSIQUE

1. récupérer les dernières modifications :

```
git pull
```

2. modifier les fichiers
3. préparer ce qu'on veut enregistrer :

```
git add fichier1 fichier2 ...
```

4. enregistrer une étape d'historique :

```
git commit -m "message clair"
```

5. envoyer vers GitLab :

```
git push
```

RÉSUMÉ DES "FLUX"

```
local  -> commit -> push -> GitLab
GitLab -> pull   -> local
```

MERGE REQUEST (MR)

Une **Merge Request**, c'est l'outil central en équipe qui permet de :

- proposer des changements (souvent depuis une branche)
- demander une **relecture** (review)
- discuter / commenter
- corriger si besoin
- puis **fusionner** dans la branche cible (souvent `main`)

Pourquoi c'est important :

- on évite d'intégrer du code "dans son coin" sans validation
- on améliore la qualité (bugs, style, compréhension)
- on garde une trace des décisions (discussion + historique)

Bon réflexe : écrire une MR avec une description courte :

- ce que ça change
- pourquoi
- comment tester (si pertinent)

GITLAB : D'AUTRES FONCTIONNALITÉS

GitLab peut aussi fournir :

- **Issues (tickets)** : organiser le travail, suivre bugs et features
- **CI/CD** : lancer automatiquement tests / builds / déploiements
- **Wiki** : documentation
- **Boards / gestion de projet** : suivi type Kanban

GitLab devient une **plateforme DevOps** autour de Git (mais Git reste l'outil de base).

CONCLUSION

CE QU'IL FAUT RETENIR

Au terme de ce cours, il faut surtout retenir que :

- **Git** permet de garder l'historique d'un projet
- il aide à **travailler proprement**, en enregistrant des étapes claires avec des **commits**
- il facilite le travail à plusieurs grâce aux **branches**, aux **fusions** et à la gestion des **conflits**
- **GitLab** ajoute une dimension **collaborative** : partage du code, relecture, discussions, tickets, intégration continue, etc.

IDÉE CLÉ

Git n’est pas seulement un outil technique :
c’est aussi un outil qui aide à mieux **organiser son travail**, à mieux **collaborer**, et à garder un projet **compréhensible dans le temps**.

EN PRATIQUE

Dans un projet réel, les bons réflexes sont souvent les mêmes :

- faire des **commits petits et clairs**
- écrire des **messages compréhensibles**
- se synchroniser régulièrement avec l'équipe
- utiliser les **branches** pour isoler son travail
- passer par une **Merge Request** pour relire et intégrer les changements

POUR LA SUITE

L'objectif n'est pas de tout maîtriser immédiatement. L'essentiel, pour commencer, est de savoir :

- créer un dépôt
- suivre l'état d'un projet
- ajouter et commiter des modifications
- envoyer et récupérer du travail
- comprendre le rôle des branches et des conflits
- utiliser GitLab comme outil de travail en équipe

Avec la pratique, ces commandes et ces habitudes deviennent naturelles.

> 📝
> Aujourd'hui, **Git** est un standard dans le développement logiciel. Il est utilisé dans la majorité des projets, que ce soit en entreprise, en open source ou dans l'enseignement.
> **GitLab** est l'une des plateformes les plus utilisées pour le travail collaboratif autour de Git, au même titre que **GitHub** ou **Bitbucket**. L'important n'est donc pas seulement de connaître un outil, mais de comprendre les **principes de collaboration** qu'il met en place.

QUESTIONS ?

Speaker notes